

Topic 2 — ITSM Integration, ChatOps, Event-Driven Automation & Self-Healing Systems

Detailed Architecture, Design, Operations and Interview/Presentation Guide

Prepared: September 2026

How to use this guide

This document expands each presentation slide into a speaker-ready explanation. It is designed for architecture discussions, technical interviews, customer presentations and production design reviews.

1. Executive Overview

Connect observability to ITSM and automation so that detected conditions become governed operational workflows. The goal is not to automate everything; it is to automate repeatable, low-risk actions while preserving human control for risky changes.

2. From Alert to Business Outcome

Signal → normalize → correlate → enrich with CMDB/topology → classify → decide → create/update ITSM record → notify ChatOps → execute runbook → validate → close or escalate.

The important design principle is closed-loop operations.

3. ITSM Integration Architecture

Use APIs/webhooks/events between monitoring and the ITSM platform. Maintain mapping for severity, service, CI, assignment group, impact, urgency, incident state and change/reference IDs.

Avoid creating one ticket for every raw event.

4. Incident Lifecycle Automation

Detection, deduplication, enrichment, incident creation, acknowledgement, diagnosis, remediation, validation and closure can each be automated to different degrees.

Keep state transitions auditable and make automation outcomes visible.

5. CMDB & Service Context

A CMDB provides ownership and relationship context: application → service → cluster → node → database → network dependency.

Good automation depends on accurate service ownership and CI relationships.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

6. ChatOps Architecture

ChatOps provides a shared operational workspace. A bot can post incidents, summarize impact, retrieve dashboards, invoke approved commands and record decisions.

Use role-based authorization, command allowlists and approval workflows for privileged actions.

7. Event-Driven Automation

Use an event bus to decouple producers and consumers. Example: AlertRaised → CorrelateEvent → IncidentCreated → ApprovalRequested → RemediationStarted → ValidationPassed → IncidentResolved.

This supports asynchronous processing and independent scaling.

8. Runbooks & Automation

A runbook should define trigger, preconditions, action, timeout, rollback, validation and escalation. Automation should be parameterized and idempotent.

Prefer small composable actions over large opaque scripts.

9. Self-Healing Systems

Self-healing means detecting a known failure pattern and automatically restoring service within predefined safety boundaries. Examples include restarting unhealthy pods, scaling workloads, clearing a stuck queue consumer or failing over a healthy replica.

10. Safety Guardrails

Guardrails include blast-radius limits, approval gates, maintenance windows, rate limits, circuit breakers, rollback, canary execution, maximum retry counts and explicit stop conditions.

If confidence is low or impact is high, route to a human.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

11. AI-Assisted Operations

AI can summarize incidents, cluster similar alerts, retrieve relevant runbooks, explain likely causes, generate investigation steps and recommend remediation.

Use deterministic policy checks around AI outputs; do not allow unrestricted autonomous production changes.

12. Example — Kubernetes Self-Healing

Alert: repeated pod crash loops. Correlation checks recent deployments, node health and dependency status. If the pattern matches a known safe condition, automation can restart or roll back according to policy, then verify readiness and error rate before closing the incident.

13. Failure Handling

Automation itself can fail. Design for timeouts, retries, dead-letter events, compensating actions and manual takeover.

Never let a remediation loop repeatedly make the same change without detecting whether the system is improving.

14. KPIs & Governance

Track automation coverage, successful remediation rate, rollback rate, false automation rate, MTTR reduction, incident recurrence, approval latency and audit completeness.

Review automation policies periodically and retire unsafe or ineffective runbooks.

15. Production Blueprint

Reference pattern: Monitoring/Event Bus → Correlation/Policy Engine → ITSM + CMDB → ChatOps → Automation Orchestrator → Cloud/Kubernetes/Network APIs → Validation → ITSM update.

Use progressive autonomy: recommend → human approve → auto-execute low-risk actions → expand only with evidence.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

Reference Areas

- ServiceNow developer documentation
- OpenTelemetry documentation
- Kubernetes documentation
- AWS EventBridge documentation
- Google Cloud Architecture Framework

Note: Product capabilities and configuration details vary by cloud/provider version. Validate implementation details against the current official product documentation before production rollout.